

Projet UML: Airwatcher

I. Introduction	2
II. Planning	3
III. Use case diagram et les descriptions détaillées	4
IV. Exigences fonctionnelles et non fonctionnelles	5
IV.1. 1. Exigences fonctionnelles (EF)	5
A. 1.1 Gestion des données et persistance	5
B. 1.2 Authentification et Contrôle d'accès	5
C. 1.3 Calculs statistiques et Qualité de l'air	5
D. 1.4 Analyse avancée et Classement (Ranking)	5
E. 1.5 Gestion du modèle incitatif et Sécurité	6
IV.2. 2. Exigences non fonctionnelles (ENF)	6
A. 2.1 Performance et Mesurabilité	6
B. 2.2 Sécurité et Architecture (Security by Design)	6
C. 2.3 Évolutivité et Extensibilité	6
D. 2.4 Robustesse et Utilisabilité	6
E. 2.5 Contraintes de déploiement	7
V. Analyse des risques de sécurité	8
VI. AirWatcher User Manual	9
VI.1. Interface de contrôle principale	9
VI.2. Commandes spécifiques aux rôles	9
A. Agence Gouvernementale	9
B. Fournisseurs & Utilisateurs privés	10
VII. Architecture logicielle	10
VII.1. Structure en quatre couches	10
A. Couche Présentation (Interface Utilisateur)	10
B. Couche Service (Logique Métier & Contrôle)	11
C. Couche Modèle (Entités de Données)	11
D. Couche d'Accès aux Données (DAO / Persistance)	11
VII.2. Justification du choix	11
VIII. Diagramme de classes	12
IX. Diagrammes de séquence	13
X. Description et pseudo-code des principaux algorithmes	14
X.1. Algorithme 1 : Calcul de la qualité de l'air en un point précis	15
X.2. Algorithme 2 : Calcul de la qualité moyenne de l'air dans une zone circulaire	16
X.3. Algorithme 3 : Classement des capteurs par similarité	17
X.4. Algorithme 4 : Détection de fraude (Capteurs non fiables)	19
X.5. Algorithme 5 : Analyse de l'impact d'un purificateur d'air	21

I. Introduction

Dans notre société actuelle, les enjeux environnementaux et sanitaires occupent une place importante. La surveillance de qualité de l'air constitue un élément essentiel pour les services publics, afin que la pollution atmosphérique n'ait pas d'impact significatif sur la santé de sa population et sur l'environnement.

Afin de mieux comprendre et de contrôler ces phénomènes, une agence gouvernementale de protection de l'environnement a installé un réseau de capteurs sur l'ensemble de son territoire. Les capteurs génèrent des données à intervalles réguliers. Ces données sont stockées dans des fichiers et elles nécessitent une exploitation efficace.

Dans ce contexte, notre projet consiste à développer une application nommée AirWatcher. Elle est destinée à surveiller et analyser la qualité de l'air sur le territoire. L'application permet de détecter les capteurs défectueux, de calculer des indicateurs comme la moyenne de qualité de l'air dans une zone donnée, de comparer les capteurs entre eux et d'estimer la qualité de l'air en un point précis. Elle prend également en compte l'impact des purificateurs d'air et la contribution des utilisateurs privés, tout en intégrant un système de détection de données frauduleuses. L'interface utilisateur est en ligne de commande et les données sont stockées dans des fichiers CSV.

Ce projet s'inscrit dans une démarche de génie logiciel et les objectifs sont multiples. Dans un premier temps, nous allons modéliser le système à l'aide de diagrammes UML. Ensuite nous concevrons une application structurée et maintenable. Pour ce faire, nous implémenterons des algorithmes efficaces. Pour finir nous testerons et validerons les fonctionnalités développées.

II. Planning

Roles:

- B3423 (Alexandre ELIAS-MENET, Tom AUTRET, Simon CABIAS)
- B3428 (Yuchen Xue, Juliette DUPREZ)

Organisation:

1. Structure du projet

Le développement de l'application AirWatcher application est organisé en plusieurs phases correspondant aux différentes séances de travaux pratiques (Lab 1 à Lab 5). Chaque phase a des objectifs précis et permet une progression structurée du projet.

2. Outils utilisés

- GitHub : gestion du code source et collaboration
- Google Drive, Docs : partage de documents
- Typst : rédaction des documents

3. Gestion du code

Le projet est géré via un dépôt GitHub partagé. Utilisation de branches pour les nouvelles fonctionnalités Validation du code via des pull requests Historique des modifications assuré par des commits réguliers

Gantt Chart:

NUMÉRO DE TÂCHE	TÂCHE	RESPONSABLE	DATE DE DÉBUT	DATE DE FIN	DURÉE (JOUR)	SEMAINE 1	SEMAINE 2	SEMAINE 3	SEMAINE 4	SEMAINE 5
1	Initialization document									
1.1	Rédiger introduction	Juliette	31/03/2026	03/04/2026	3					
1.2	Effectuer planning	Juliette / Tom	31/03/2026	03/04/2026	3					
2	Document de spécification des exigences									
2.1	Diagramme des cas d'usage	Simon	31/03/2026	03/04/2026	3					
2.2	Spécifications fonctionnelles et non fonctionnelles	Yuchen	31/03/2026	03/04/2026	3					
2.3	Analyse des risques de sécurité	Alexandre	31/03/2026	03/04/2026	3					
2.4	Manuel utilisateur	Tom	31/03/2026	19/05/2026	49					
3	Analyse et Documents d'architecture									
3.1	Architecture / Description et arguments de ce choix	Juliette	21/04/2026	27/04/2026	6					
3.2	Diagramme de classe	Alexandre/Tom	21/04/2026	27/04/2026	6					
3.3	Diagramme de séquence	Juliette	21/04/2026	27/04/2026	6					
3.4	Description / Pseudo code des principaux algorithmes	Yuchen	21/04/2026	04/05/2026	13					
4	Développement de l'application									
4.1	Ecrire code source	Groupe	28/04/2026	19/05/2026	21					
4.2	Ecrire guide pour créer l'exécutable	Groupe	05/05/2026	19/05/2026	14					
4.3	Tests	Groupe	05/05/2026	19/05/2026	14					
5	Présentation et demo									
5.1	Présentation de la conception de l'app	Groupe	26/05/2026	26/05/2026	0					
5.2	Démo des fonctionnalités implémentées	Groupe	26/05/2026	26/05/2026	0					
5.3	Évaluation des performances des algo executés	Groupe	26/05/2026	26/05/2026	0					
5.4	Passage en revue du code d'un algo clé	Groupe	26/05/2026	26/05/2026	0					

Fig. 1 : Projet de capteur - Gantt Chart

III. Use case diagram et les descriptions détaillées

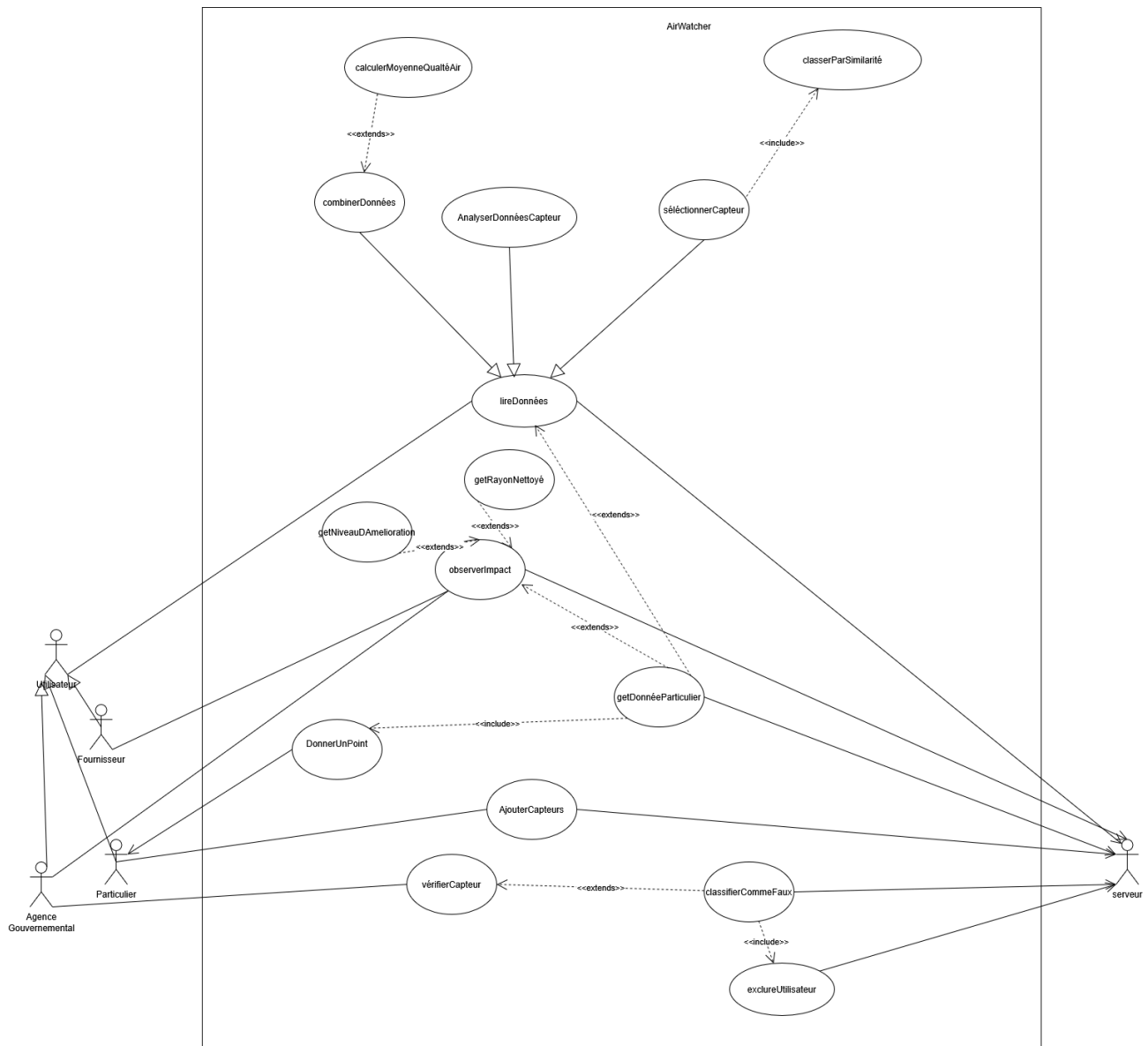


Fig. 2 : Projet de capteur - UseCaseDiagram

IV. Exigences fonctionnelles et non fonctionnelles

IV.1. 1. Exigences fonctionnelles (EF)

Cette section détaille l'ensemble des fonctionnalités que le système doit implémenter. Chaque exigence est atomique, mesurable et directement liée à un cas d'utilisation du système.

A. 1.1 Gestion des données et persistance

- **EF-101 (Chargement initial)** : Le système doit charger automatiquement en mémoire l'intégralité des données des fichiers CSV locaux (users.csv, sensors.csv, cleaners.csv, measurements.csv) dès son initialisation.
- **EF-102 (Ajout continu)** : Le système doit permettre l'insertion de nouvelles mesures et de nouveaux utilisateurs en cours d'exécution, et persister immédiatement ces modifications dans les fichiers CSV correspondants.

B. 1.2 Authentification et Contrôle d'accès

- **EF-201 (Authentification)** : Le système doit fournir un mécanisme de connexion sécurisé demandant un identifiant et un mot de passe pour identifier le rôle de l'utilisateur (GovernmentAgency, Provider, PrivateUser).
- **EF-202 (Menus dynamiques)** : L'interface CLI doit restreindre l'affichage des commandes disponibles en fonction des privilèges du rôle identifié (ex: le menu de bannissement de capteur est invisible pour un PrivateUser).

C. 1.3 Calculs statistiques et Qualité de l'air

- **EF-301 (Conversion ATMO)** : Le système doit être capable de convertir un ensemble de concentrations de polluants (O_3 , NO_2 , SO_2 , PM_{10}) en un indice ATMO entier compris entre 1 (Très bon) et 10 (Exécrable), basé sur le sous-indice le plus défavorable.
- **EF-302 (Moyenne de zone spatio-temporelle)** : Le système doit calculer l'indice de qualité de l'air moyen pour une zone définie par des coordonnées géographiques (latitude, longitude) et un rayon (r), soit à un instant t précis, soit sur un intervalle de temps (TimeRange).
- **EF-303 (Estimation par interpolation)** : Le système doit estimer la qualité de l'air à n'importe quelles coordonnées (lat, lon) dépourvues de capteur en appliquant une méthode d'interpolation spatiale pondérée par l'inverse du carré de la distance ($\frac{1}{d^2}$) par rapport aux capteurs fiables environnants.

D. 1.4 Analyse avancée et Classement (Ranking)

- **EF-401 (Classement par similarité)** : Le système doit permettre de sélectionner un capteur de référence et de générer une liste triée de tous les autres capteurs du réseau, du plus similaire au moins similaire, sur une période temporelle donnée.
- **EF-402 (Analyse d'impact des purificateurs)** : Le système doit évaluer l'efficacité d'un purificateur d'air en calculant son rayon d'action réel et son pourcentage d'amélioration de l'air, en comparant les données de la zone pendant son fonctionnement avec une période témoin antérieure de même durée.

E. 1.5 Gestion du modèle incitatif et Sécurité

- **EF-501 (Attribution automatique de points)** : Le système doit incrémenter le solde de points d'un PrivateUser de un (+1) point chaque fois que les données de son capteur privé sont lues et intégrées dans un calcul statistique global.
- **EF-502 (Détection d'anomalies de voisinage)** : Le système doit analyser la cohérence des données d'un capteur privé en comparant ses relevés avec la moyenne des capteurs voisins dans un rayon donné.
- **EF-503 (Bannissement et Sanction)** : Lorsqu'une fraude ou une défaillance est détectée sur un capteur par l'Agence, le système doit basculer l'état du capteur à `isReliable = false`, l'exclure des futurs calculs d'interpolation, et geler immédiatement l'attribution de points pour son propriétaire.

IV.2. 2. Exigences non fonctionnelles (ENF)

Les exigences non fonctionnelles décrivent les contraintes techniques, de performance, de sécurité et d'ergonomie imposées au système.

A. 2.1 Performance et Mesurabilité

- **ENF-101 (Chronométrage des algorithmes)** : Le système doit mesurer systématiquement le temps d'exécution CPU de chaque algorithme majeur (interpolation, similarité, rayon d'action) et l'afficher explicitement dans la console en millisecondes (ms).
- **ENF-102 (Temps de réponse)** : Les opérations de filtrage et de calcul sur les structures en mémoire ne doivent pas excéder un temps de traitement de 500 millisecondes pour un volume standard de données.

B. 2.2 Sécurité et Architecture (Security by Design)

- **ENF-201 (Isolation des données)** : Le système doit interdire tout accès direct ou manipulation des fichiers CSV par l'utilisateur final. Toute lecture/écriture doit impérativement transiter par la couche DAO (CSVDataManager) et être validée par le SecurityService.

C. 2.3 Évolutivité et Extensibilité

- **ENF-301 (Ajout d'entités sans recompilation)** : Le modèle de données et la couche d'accès doivent être conçus pour supporter l'ajout de nouvelles lignes de capteurs ou d'utilisateurs dans les fichiers CSV sans nécessiter de modification ou de recompilation du code source de l'application.

D. 2.4 Robustesse et Utilisabilité

- **ENF-401 (Tolérance aux pannes de saisie)** : Le système doit intercepter toutes les erreurs de formatage dans la console (ex: saisie d'une chaîne de caractères au lieu d'une coordonnée double, rayon négatif) sans crasher, et lever des exceptions claires pour inviter l'utilisateur à corriger sa saisie.

E. 2.5 Contraintes de déploiement

- **ENF-501 (Indépendance réseau) :** L'application doit s'exécuter de manière totalement autonome en local (Stand-alone CLI). L'intégration de protocoles réseaux, de sockets ou de bases de données distantes est explicitement déclarée hors périmètre pour cette version.

V. Analyse des risques de sécurité

Atout	Vulnérabilité	Attaque	Risque	Contre-mesure
Données des capteurs	Absence de vérification des données	Injection de fausses données	Données incorrectes	Détection d'anomalies + exclusion des capteurs suspects
Données des capteurs	Absence de contrôle d'intégrité	Modification des valeurs	Statistiques faussées	Validation + contrôle d'intégrité
Données utilisateurs	Mots de passe faibles	Deviner mot de passe	Accès non autorisé	Mots de passe forts + hachage
Données utilisateurs	Pas de gestion des droits	Accès à toutes les données	Fuite d'informations	Gestion des rôles
Fichiers CSV	Accès non sécurisé	Modification/suppression	Perte de données	Restriction d'accès + sauvegardes
Système (serveur)	Point unique de défaillance	Attaque Dos	Indisponibilité	Redondance + sauvegardes
Système	Pas de validation des entrées	Injection de données	Crash ou corruption	Validation stricte des entrées
Application	Absence de logs	Actions non traçables	Impossible d'identifier l'attaque	Journalisation
Données sensibles	Pas de chiffrement	Interception	Fuite d'informations	Chiffrement

VI. AirWatcher User Manual

VI.1. Interface de contrôle principale

L'utilisateur interagit avec le système via un menu principal.

— MENU PRINCIPAL AIRWATCHER —

- [1] Analyser un capteur
 - [2] Calculer la qualité de l'air dans une zone à un instant donné
 - [3] Calculer la qualité de l'air dans une zone sur une période donnée
 - [3] Comparer des capteurs par similarité
 - [4] Calculer la qualité de l'air globale à une position et date données
 - [5] Consulter les purificateurs
 - [6] Quitter
-

Note : Entrez le numéro correspondant pour exécuter la commande.

VI.2. Commandes spécifiques aux rôles

A. Agence Gouvernementale

Ce rôle, en plus des premières fonctionnalités proposées, dispose d'un accès complet pour la maintenance de l'intégrité du système.

— MENU AGENCE GOUVERNEMENTALE —

- [6] Vérifier si un capteur est défectueux
 - [7] Identifier les comportements frauduleux
 - [8] Identifier les données corrompues
 - [9] Restaurer la base de données
 - [10] Quitter
-

B. Fournisseurs & Utilisateurs privés

Les fonctionnalités de consultation sont partagées pour favoriser la transparence.

- **Fournisseurs (Providers) :** Évaluent l'efficacité des purificateurs via `view_cleaner_impact`.

— MENU FOURNISSEUR —

- [6] Voir l'impact d'un purificateur
 - [7] Comparer les capteurs par similarité
 - [8] Analyser le rayon de purification
 - [9] Consulter les statistiques de zone
 - [10] Quitter
-

- **Particuliers :** Accèdent au solde de récompenses via `view_points`.

— MENU PARTICULIERS —

- [6] Consulter mon solde de points
 - [7] Calculer la moyenne AQI de ma zone
 - [8] Comparer les capteurs du voisinage
 - [9] Saisir des mesures de capteur privé
 - [10] Voir l'historique de mes contributions
 - [11] Quitter
-

VII. Architecture logicielle

Pour répondre aux exigences de modularité, de performance et de sécurité, nous avons adopté une **architecture en couches à quatre niveaux**. Ce découpage permet une stricte séparation des responsabilités, isole totalement les structures de données de la logique applicative, et facilite l'évolution du système.

VII.1. Structure en quatre couches

A. Couche Présentation (Interface Utilisateur)

- Implémente l'interface en ligne de commande (CLI) adaptée aux privilèges de chaque rôle (Agence, Fournisseur, Particulier).
- Capture les entrées utilisateur et affiche les résultats sans contenir aucune logique métier ni structure de données.

B. Couche Service (Logique Métier & Contrôle)

- Agit comme le centre de pilotage de l'application via le contrôleur principal (AirWatcherSystem), effectuant l'interface directe avec la présentation.
- Centralise la logique algorithmique à travers des modules spécialisés (StatisticsService, SecurityService, DataService).
- Intègre la mesure de performance des traitements en millisecondes et orchestre l'application des règles de gestion (comme la détection et l'exclusion des fraudes).

C. Couche Modèle (Entités de Données)

- Regroupe les classes métiers pures représentant le domaine de l'application (User et ses spécialisations, Sensor, Measurement, AirCleaner, Attribute, TimeRange).
- Contient uniquement la structure des objets et leurs données (attributs, getters/setters, calculs géométriques locaux simples comme la distance), sans aucune logique de persistance ni d'algorithme global.

D. Couche d'Accès aux Données (DAO / Persistance)

- Implémentée par le CSVDataManager, elle assure l'unique passerelle de lecture et d'écriture avec les fichiers physiques (sensors.csv, measurements.csv, etc.).
- Abstrait techniquement la structure des fichiers CSV pour instancier et charger les objets de la couche Modèle afin qu'ils soient exploitables par la couche Service.

VII.2. Justification du choix

- **Sécurité** : Garantit que les utilisateurs n'ont aucun accès direct aux fichiers de données.
- **Maintenabilité** : Permet de modifier le format de stockage ou l'interface utilisateur sans impacter les algorithmes de calcul.
- **Testabilité** : Facilite la réalisation de tests unitaires sur les fonctionnalités critiques (moyennes, rankings) indépendamment de l'interface.

VIII. Diagramme de classes

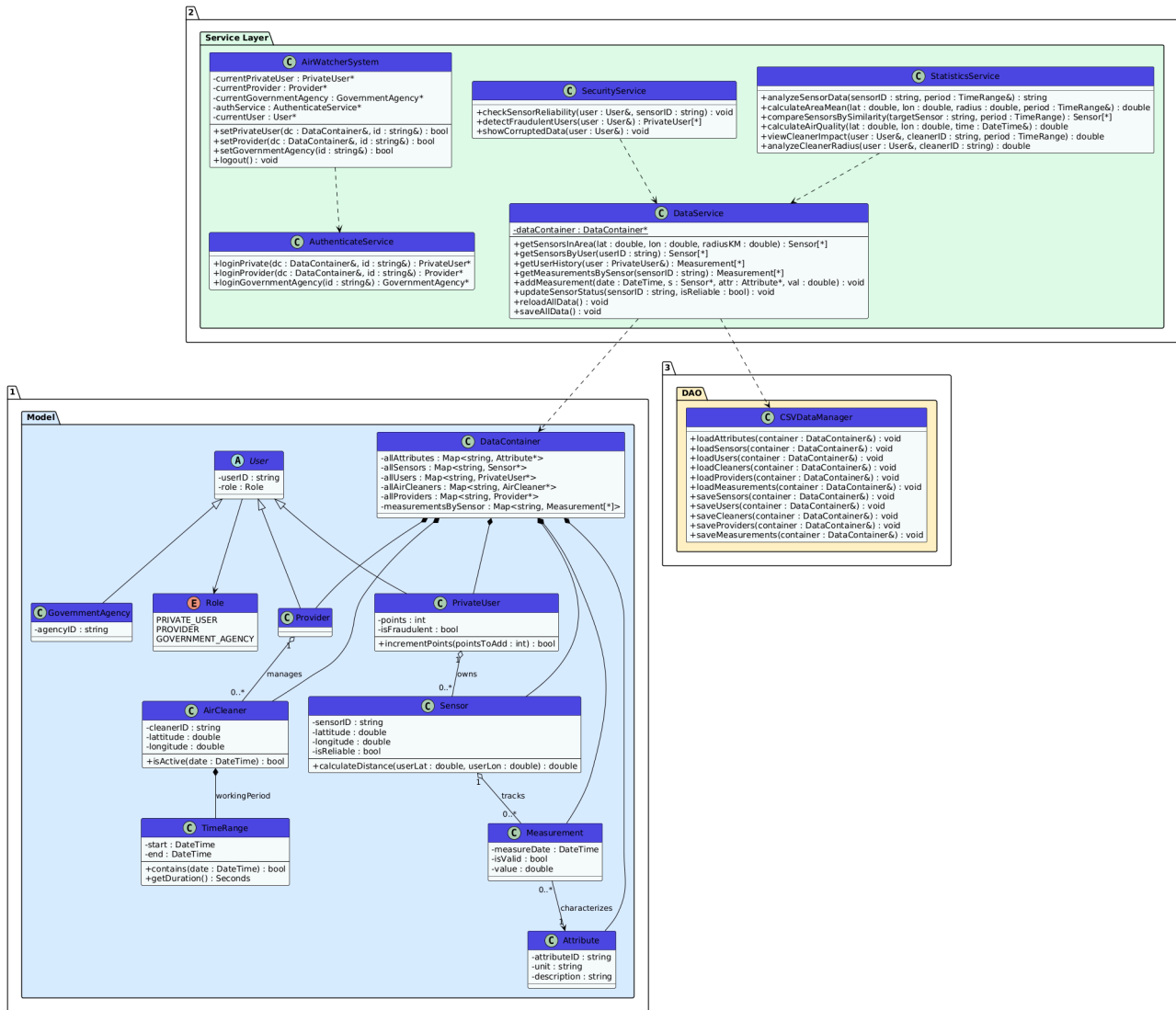


Fig. 3 : Diagramme de classes

IX. Diagrammes de séquence

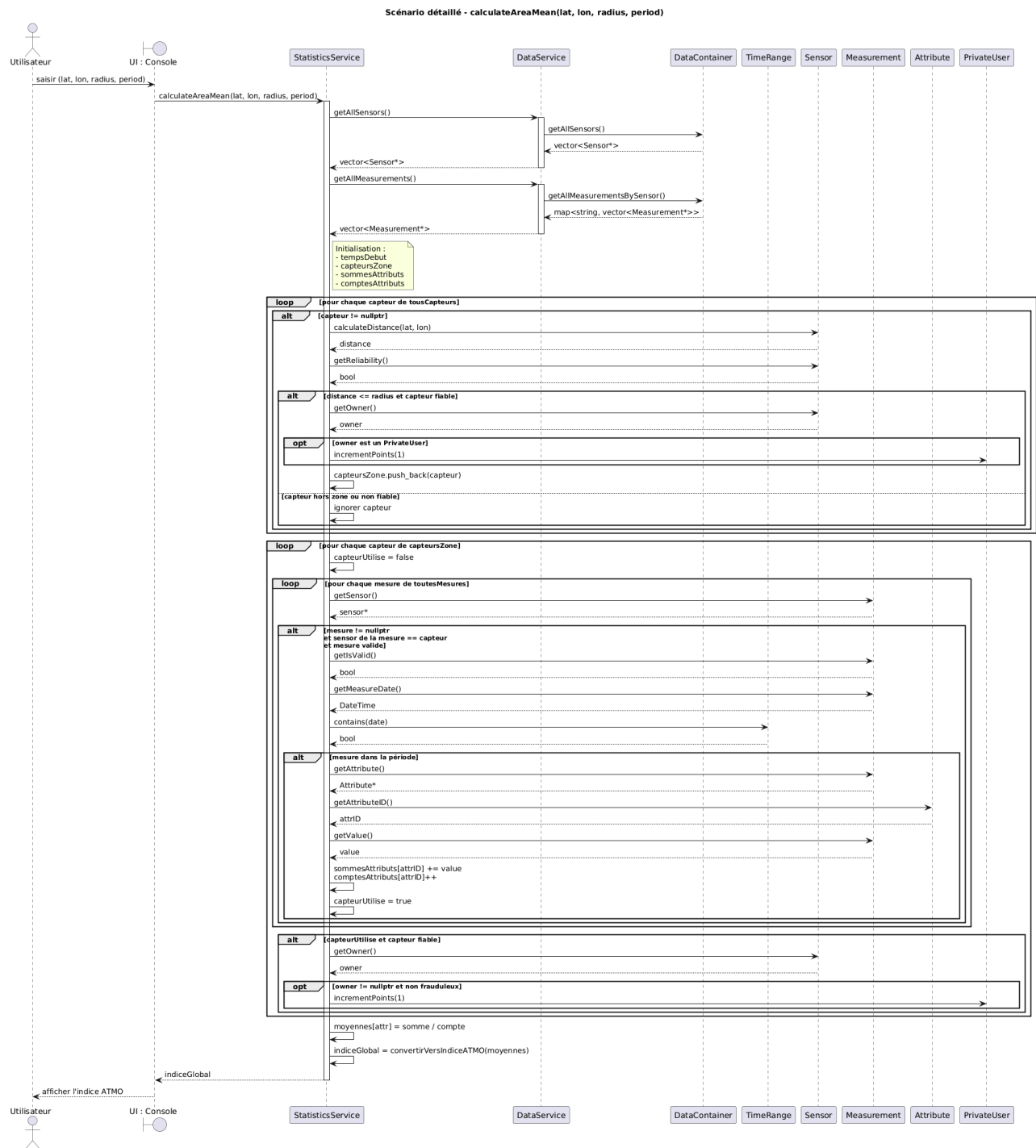


Fig. 4 : Diagramme de séquence pour le calcul de la moyenne AQI dans une zone

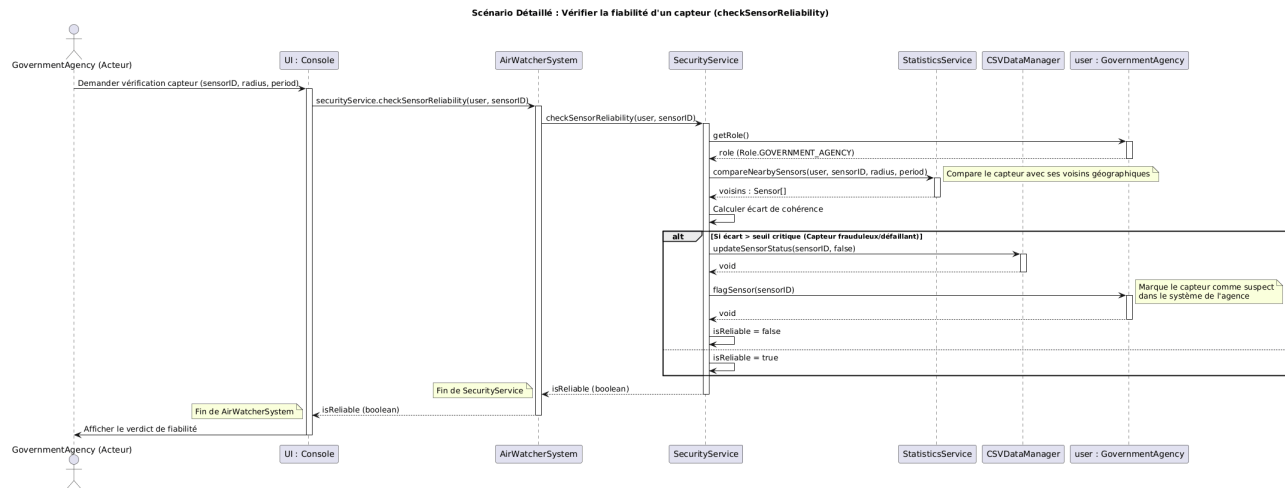


Fig. 5 : Diagramme de séquence pour vérifier la fiabilité d'un capteur

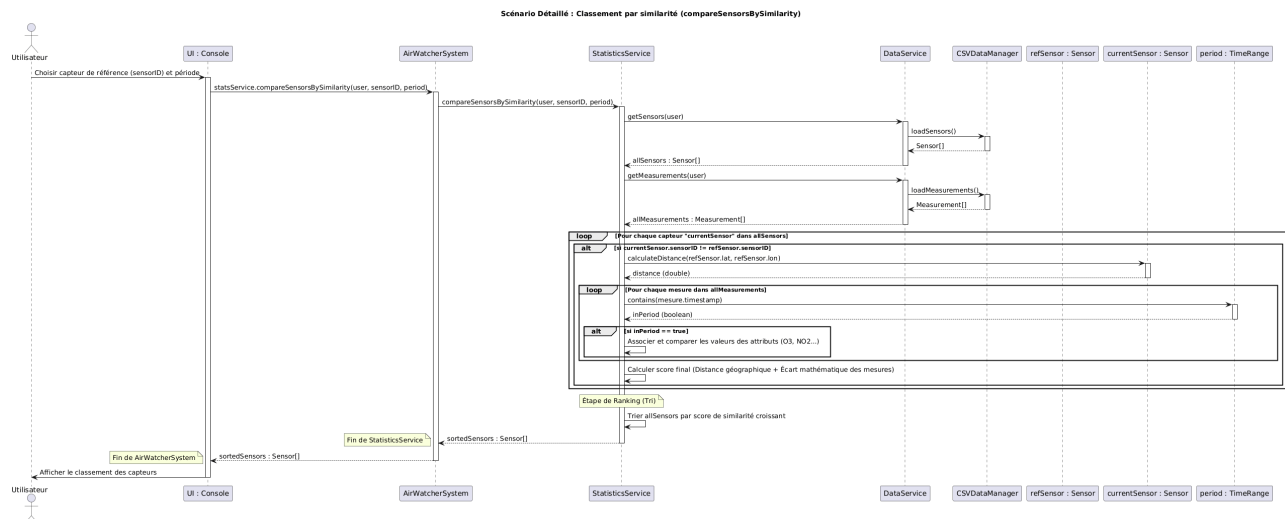


Fig. 6 : Diagramme de séquence pour la classement des capteurs par similarité

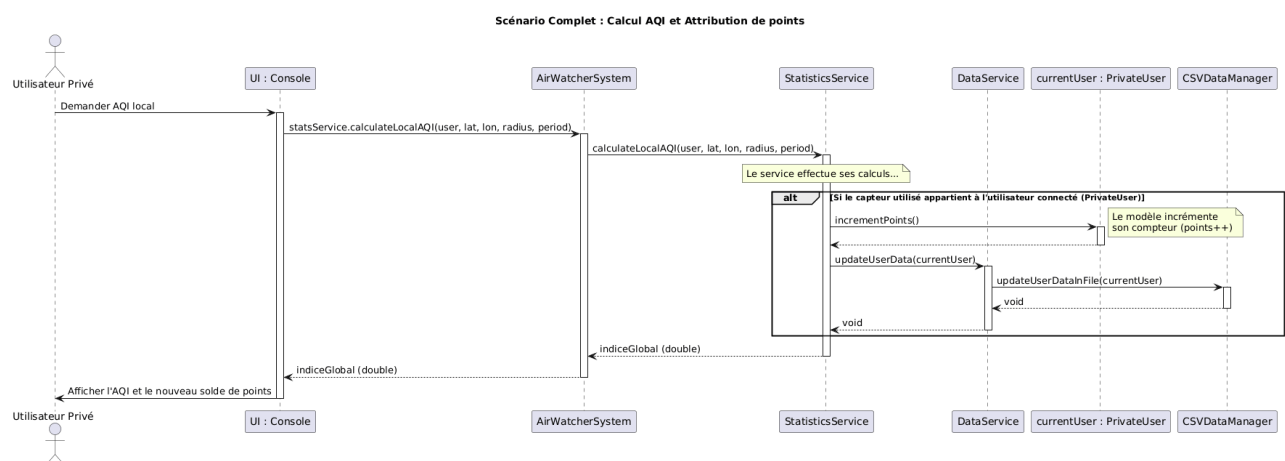


Fig. 7 : Diagramme de séquence pour l'attribution des points

X. Description et pseudo-code des principaux algorithmes

Cette section détaille la logique interne des algorithmes les plus critiques du système AirWatcher. Pour répondre aux exigences non fonctionnelles de performance, chaque

algorithme intègre un mécanisme de chronométrage mesurant son temps d'exécution en millisecondes. Les signatures et types de retour respectent strictement le diagramme de classes.

X.1. Algorithme 1 : Calcul de la qualité de l'air en un point précis

- **Objectif** : Estimer l'indice de qualité de l'air (AQI) à une position géographique exacte où aucun capteur n'est forcément présent, en utilisant une interpolation spatiale des capteurs environnants.
- **Classe et Méthode** : `StatisticsService.calculateAirQuality(lat: double, lon: double, time: DateTime) : double`
- **Logique de l'algorithme** :
 1. Récupération de l'ensemble des capteurs et des mesures globaux du système.
 2. Filtrage pour ne conserver que les capteurs fiables et les mesures valides correspondant exactement à l'instant demandé.
 3. Application d'une pondération spatiale par l'inverse du carré de la distance ($\frac{1}{d^2}$) pour donner plus d'importance aux capteurs les plus proches du point ciblé.
 4. Mécanisme d'incitation (Gamification) : attribution d'un point de récompense aux utilisateurs privés si, et seulement si, les données de leurs capteurs ont été effectivement exploitées lors de ce calcul.
 5. Agrégation pondérée des valeurs des différents polluants et conversion finale en indice ATMO

Pseudo-code :

```
FONCTION calculateAirQuality(lat, lon, time):
    tempsDebut = obtenirTempsActuel()

    // Récupération des données globales du système
    tousCapteurs = DataService.getAllSensors()
    toutesMesures = DataService.getAllMeasurements()

    sommesPonderees = DICTIONNAIRE_VIDE
    sommePoids = DICTIONNAIRE_VIDE

    // Étape 1 : Interpolation spatiale et filtrage des capteurs
    POUR CHAQUE capteur DANS tousCapteurs:
        SI capteur.isReliable == VRAI ALORS:
            distance = capteur.calculateDistance(lat, lon)

    // Éviter la division par zéro si le point correspond exactement à un capteur
    poids = (distance < 0.1) ? 100 : 1 / (distance * distance)
    capteurUtilise = FAUX

    // Étape 2 : Filtrage temporel, vérification de validité et pondération
    POUR CHAQUE mesure DANS toutesMesures:
        SI mesure.sensorID == capteur.sensorID ET mesure.isValid == VRAI ET
        mesure.time == time ALORS:
            sommesPonderees[mesure.attributeID] += (mesure.value * poids)
            sommePoids[mesure.attributeID] += poids
```

```
capteurUtilise = VRAI

// Étape 3 : Application du système de récompense pour les
contributeurs
SI capteurUtilise == VRAI ALORS:
    owner = capteur.getOwner()
    SI owner EST UN PrivateUser ET owner.isFraudulent == FAUX ALORS:
        owner.incrementPoints(1)

// Étape 4 : Calcul des moyennes pondérées par polluant
moyennesEstimees = DICTIONNAIRE_VIDE
POUR CHAQUE attrID DANS sommesPonderees:
    SI sommePoids[attrID] > 0 ALORS:
        moyennesEstimees[attrID] = sommesPonderees[attrID] / sommePoids[attrID]

// Étape 5 : Détermination de l'indice final
indiceGlobal = convertirVersIndiceATMO(moyennesEstimees)

tempsFin = obtenirTempsActuel()
imprimer("Temps d'exécution calculateAirQuality : " + (tempsFin - tempsDebut) +
" ms")
```

X.2. Algorithme 2 : Calcul de la qualité moyenne de l'air dans une zone circulaire

- **Objectif :** Évaluer la qualité globale de l'air d'une zone géographique définie par un centre et un rayon, durant une période spécifique.
- **Classe et Méthode :** StatisticsService.calculateAreaMean(lat: double, lon: double, radius: double, period: TimeRange): double
- **Logique de l'algorithme :**
 1. Récupération de l'ensemble des capteurs et des mesures globaux du système.
 2. Filtrage spatial pour isoler les capteurs situés dans le rayon spécifié et qui sont considérés comme fiables.
 3. Filtrage temporel et vérification de validité des mesures associées à ces capteurs de la zone.
 4. Mécanisme d'incitation (Gamification) : attribution d'un point de récompense aux utilisateurs privés dont les capteurs fiables ont fourni des données exploitées pour ce calcul.
 5. Calcul de la moyenne des valeurs pour chaque polluant présent.
 6. Conversion des moyennes obtenues en un indice ATMO global.

Pseudo-code :

```
FONCTION calculateAreaMean(lat, lon, radius, period):
    tempsDebut = obtenirTempsActuel()

    // Accès aux données globales via DataService
    tousCapteurs = DataService.getAllSensors()
    toutesMesures = DataService.getAllMeasurements()

    capteursZone = LISTE_VIDE
```



```
// Étape 1 : Filtrage spatial et vérification de fiabilité
POUR CHAQUE capteur DANS tousCapteurs:
    distance = capteur.calculateDistance(lat, lon)
    SI distance <= radius ET capteur.isReliable == VRAI ALORS:
        AJOUTER capteur À capteursZone

// Étape 2 : Agrégation des mesures correspondantes
sommesAttributs = DICTIONNAIRE_VIDE
comptesAttributs = DICTIONNAIRE_VIDE

POUR CHAQUE capteur DANS capteursZone:
    capteurUtilise = FAUX
    POUR CHAQUE mesure DANS toutesMesures:
        // Filtrage d'appartenance, de validité et temporel
        SI mesure.sensorID == capteur.sensorID ET mesure.isValid == VRAI ET
period.contains(mesure.timestamp) ALORS:
            sommesAttributs[mesure.attributeID] += mesure.value
            comptesAttributs[mesure.attributeID] += 1
            capteurUtilise = VRAI

// Étape 3 : Application du système de récompense
SI capteurUtilise == VRAI ET capteur.isReliable == VRAI ALORS:
    owner = capteur.getOwner()
    SI owner EST UN PrivateUser ET owner.isFraudulent == FAUX ALORS:
        owner.incrementPoints(1)

// Étape 4 : Calcul des moyennes par polluant
moyennes = DICTIONNAIRE_VIDE
POUR CHAQUE attribut DANS sommesAttributs:
    SI comptesAttributs[attribut] > 0 ALORS:
        moyennes[attribut] = sommesAttributs[attribut] /
comptesAttributs[attribut]

// Étape 5 : Conversion finale
indiceGlobal = convertirVersIndiceATM0(moyennes)

tempsFin = obtenirTempsActuel()
imprimer("Temps d'exécution calculateAreaMean : " + (tempsFin - tempsDebut) + "
ms")

RETOURNER indiceGlobal
```

X.3. Algorithme 3 : Classement des capteurs par similarité

- **Objectif** : Identifier les capteurs ayant une qualité d'air similaire en comparant leurs données à celles d'un capteur de référence sur une période donnée.
- **Classe et Méthode** : `StatisticsService.compareSensorsBySimilarity(targetSensor: string, period: TimeRange): vector`
- **Logique de l'algorithme** :
 1. Récupération globale de tous les capteurs et mesures du système.

2. Extraction des mesures spécifiques au capteur cible sur la période demandée. Attribution d'un point de récompense au propriétaire s'il s'agit d'un utilisateur privé.
3. Pour chaque autre capteur considéré comme fiable :
 - Extraction de ses mesures sur la même période et récompense de son propriétaire.
 - Calcul de l'écart absolu moyen par rapport aux données du capteur cible pour les mêmes horodatages et attributs.
4. Tri des capteurs évalués par ordre croissant d'écart moyen (un écart plus petit signifiant une plus grande similarité).

Pseudo-code :

```
FONCTION compareSensorsBySimilarity(targetSensor, period):
    tempsDebut = obtenirTempsActuel()

    // Récupération des données globales
    tousCapteurs = DataService.getAllSensors()
    toutesMesures = DataService.getAllMeasurements()

    // Étape 1 : Extraction des données cibles et récompense
    donneesCible = LISTE_VIDE
    POUR CHAQUE mesure DANS toutesMesures:
        SI mesure.sensorID == targetSensor ET period.contains(measure.timestamp) ET
        mesure.isValid == VRAI ALORS:
            AJOUTER mesure À donneesCible

    SI donneesCible N'EST PAS VIDE ALORS:
        owner = obtenirProprietaire(targetSensor)
        SI owner EST UN PrivateUser ET owner.isFraudulent == FAUX ALORS:
            owner.incrementPoints(1)

    scoresSimilarite = DICTIONNAIRE_VIDE // {sensorID: ecartMoyen}

    // Étape 2 : Comparaison avec les autres capteurs
    POUR CHAQUE capteur DANS tousCapteurs:
        SI capteur.sensorID == targetSensor OU capteur.isReliable == FAUX ALORS:
            CONTINUER

        donneesAComparer = LISTE_VIDE
        POUR CHAQUE mesure DANS toutesMesures:
            SI mesure.sensorID == capteur.sensorID ET
            period.contains(measure.timestamp) ET mesure.isValid == VRAI ALORS:
                AJOUTER mesure À donneesAComparer

        SI donneesAComparer N'EST PAS VIDE ALORS:
            owner = capteur.getOwner()
            SI owner EST UN PrivateUser ET owner.isFraudulent == FAUX ALORS:
                owner.incrementPoints(1)

    // Étape 3 : Calcul de l'écart moyen
```

```
differenceTotale = 0.0
pointsCommuns = 0

POUR CHAQUE mesureCible DANS donneesCible:
    mesureCorrespondante = trouverMesure(donneesAComparer,
mesureCible.timestamp, mesureCible.attributeID)
    SI mesureCorrespondante EXISTE ALORS:
        differenceTotale += VALEUR_ABSOLUE(mesureCible.value -
mesureCorrespondante.value)
        pointsCommuns += 1

SI pointsCommuns > 0 ALORS:
    ecartMoyen = differenceTotale / pointsCommuns
    scoresSimilarite[capteur.sensorID] = ecartMoyen

// Étape 4 : Tri par écart croissant (le plus petit écart en premier)
listeTrie = TRIER_CAPTEURS_PAR_SCORE(scoresSimilarite, CROISSANT)

tempsFin = obtenirTempsActuel()
imprimer("Temps d'exécution : " + (tempsFin - tempsDebut) + " ms")

RETOURNER listeTrie
```

X.4. Algorithme 4 : Détection de fraude (Capteurs non fiables)

- **Objectif :** Analyser le comportement du capteur d'un utilisateur pour s'assurer qu'il ne falsifie pas les données, action réservée aux agences gouvernementales.
- **Classe et Méthode :** SecurityService.checkSensorReliability(user: User, targetSensorID: string) : void
- **Logique de l'algorithme :**
 1. Vérification des droits d'accès : seule une agence gouvernementale est autorisée à lancer cette analyse.
 2. Récupération du capteur cible. S'il est déjà marqué comme non fiable, l'analyse s'arrête.
 3. Identification des capteurs voisins fiables situés dans un rayon de 50 km autour de la cible.
 4. Extraction de toutes les mesures valides de ces voisins.
 5. Pour chaque mesure du capteur ciblé, calcul d'une valeur de référence (moyenne des mesures des voisins au même instant pour le même polluant).
 6. Si l'écart entre la valeur ciblée et la référence dépasse un seuil de tolérance, la mesure est invalidée et comptabilisée comme anomalie.
 7. Si le taux d'anomalies global dépasse un seuil critique (ex: 50%), le capteur est définitivement marqué comme frauduleux dans le système.

Pseudo-code :

```
FONCTION checkSensorReliability(user, targetSensorID):
    tempsDebut = obtenirTempsActuel()

    // Étape 1 : Contrôle d'accès
```

```
SI user.role != GOVERNMENT_AGENCY ALORS:
    imprimer("Erreur : Accès non autorisé.")
    RETOURNER

capteurAAAnalyser = OBTENIR_CAPTEUR(targetSensorID)
SI capteurAAAnalyser.isReliable == FAUX ALORS:
    RETOURNER // Déjà identifié comme frauduleux

mesuresCible = OBTENIR_MESURES_CAPTEUR(targetSensorID)
tousCapteurs = OBTENIR_TOUS_LES_CAPTEURS()

// Étape 2 : Identification du voisinage (50 km)
capteursVoisinsFiables = LISTE_VIDE
POUR CHAQUE capteur DANS tousCapteurs:
    SI capteur.isReliable == VRAI ET capteur.sensorID != targetSensorID ALORS:
        distance = capteur.calculateDistance(capteurAAAnalyser.latitude,
capteurAAAnalyser.longitude)
        SI distance <= 50.0 ALORS:
            AJOUTER capteur À capteursVoisinsFiables

SI capteursVoisinsFiables EST VIDE ALORS:
    capteurAAAnalyser.isReliable = VRAI
    RETOURNER

// Extraction des mesures fiables du voisinage
mesuresFiablesVoisins = EXTRAIRE_MESURES_VALIDES(capteursVoisinsFiables)

// Étape 3 : Analyse comparative
anomaliesDetectees = 0
totalMesures = 0
SEUIL_TOLERANCE = 0.30
SEUIL_FRAUDE = 0.50

POUR CHAQUE mesure DANS mesuresCible:
    sommValeurs = 0.0
    compteMesures = 0

    POUR CHAQUE mesureVoisin DANS mesuresFiablesVoisins:
        SI mesureVoisin.date == mesure.date ET mesureVoisin.attribut ==
mesure.attribut ALORS:
            sommValeurs += mesureVoisin.value
            compteMesures += 1

SI compteMesures > 0 ALORS:
    valeurReference = sommValeurs / compteMesures
    totalMesures += 1
    ecartAbsolu = VALEUR_ABSOLUE(mesure.value - valeurReference)

// Étape 4 : Détection d'anomalie
SI ecartAbsolu > (valeurReference * SEUIL_TOLERANCE) ALORS:
    mesure.isValid = FAUX // Invalidation de la mesure fausse
    anomaliesDetectees += 1
```

```
// Étape 5 : Verdict final
estFiable = VRAI
SI totalMesures > 0 ALORS:
    tauxAnomalies = anomaliesDetectees / totalMesures
    SI tauxAnomalies > SEUIL_FRAUDE ALORS:
        estFiable = FAUX
        capteurAnalyser.isReliable = FAUX // Mise à jour dans le conteneur de
données

tempsFin = obtenirTempsActuel()
imprimer("Temps d'exécution : " + (tempsFin - tempsDebut) + " ms")
```

X.5. Algorithme 5 : Analyse de l'impact d'un purificateur d'air

- **Objectif** : Mesurer l'efficacité d'un purificateur d'air (AirCleaner) en comparant la qualité de l'air dans sa zone d'effet (rayon de 5 km) avant et après son activation au sein d'une période spécifiée.
- **Classe et Méthode** : `StatisticsService.viewCleanerImpact(cleanerID: string, period: TimeRange): double`
- **Logique de l'algorithme** :
 1. Localisation du purificateur ciblé via la couche de données.
 2. Division de la période d'étude globale en deux segments temporels : «Avant» l'activation et «Après» l'activation, en se basant sur l'heure de démarrage du purificateur.
 3. Appel de l'algorithme de calcul de moyenne de zone (`calculateAreaMean`) pour ces deux sous-périodes sur le rayon d'action défini.
 4. Calcul du différentiel de qualité d'air (un impact positif signifiant une baisse de la pollution).

Pseudo-code :

```
FONCTION viewCleanerImpact(cleanerID, period):
    tempsDebut = obtenirTempsActuel()

    cleaner = DataService.getCleanerById(cleanerID)
    rayonEffet = 5.0 // Rayon par défaut fixé dans l'implémentation

    // Séparation des périodes
    periodeAvant = TimeRange(period.start, cleaner.startTime)
    periodeApres = TimeRange(cleaner.startTime, period.end)

    // Calcul des qualités d'air (utilisation de l'algorithme de zone)
    qualiteAvant = calculateAreaMean(cleaner.lat, cleaner.lon, rayonEffet,
periodeAvant)
    qualiteApres = calculateAreaMean(cleaner.lat, cleaner.lon, rayonEffet,
periodeApres)

    // Calcul de l'impact (différence absolue)
    impactRelatif = qualiteAvant - qualiteApres
```

```
tempsFin = obtenirTempsActuel()  
imprimer("Analyse effectuée en : " + (tempsFin - tempsDebut) + " ms")  
  
RETOURNER impactRelatif
```